

JobPilot

Smart Job Matcher -- Technical Brief

BAX-423 Big Data | Spring 2026 | Dr. Rahul Makhijani | UC Davis MSBA

1. System Architecture

JobPilot is a single-process Flask application that ingests real job postings, matches them to an uploaded resume using hybrid sparse + dense retrieval, re-ranks candidates through a multi-stage pipeline, adapts to user feedback via the Rocchio algorithm, and lets users download results or generate a tailored resume.

Layer	Technology	Purpose
Web Framework	Flask 3.1 + Unicorn	HTTP routing, sessions, template rendering
Database	SQLite + SQLAlchemy	Jobs, sessions, feedback -- zero-setup file DB
Dense Retrieval	FAISS IndexFlatIP	ANN search over 8,280 job vectors (384-dim)
Sparse Retrieval	BM25Okapi (rank-bm25)	Keyword candidate pool, top-500 per query
Embedding Model	BAAI/bge-small-en (ONNX)	384-dim vectors via fastembed; no PyTorch
Adaptive Learn.	Rocchio Algorithm	Query vector updated from user feedback
Streaming	APScheduler + Adzuna API	Background polling; SHA-256 deduplication
Resume Gen.	Claude API / Template fallback	Tailors resume to selected job description
Deployment	Render (Docker)	https://jobpilot-fhvp.onrender.com

Pipeline Flow

PDF upload -> text extraction -> 384-dim query vector -> Stage 1: hard filters (dealbreakers, salary, seniority, experience, US-only) -> Stage 2: BM25 top-500 + FAISS ANN top-500 merged -> Stage 3: hybrid re-rank (0.7 x dense + 0.3 x BM25 + 0.1 x skill overlap) -> Rocchio update per feedback round.

Live app: <https://jobpilot-fhvp.onrender.com> | GitHub: <https://github.com/Bodazhu/jobpilot>

2. BAX-423 Technique Choices & Benchmark Results

Technique 1 -- BM25 (Sparse Retrieval) [Text Mining / IR lectures]

- * TF-IDF probabilistic ranking over job title + company + skills + description[:300].
- * Why: Exact keyword matching excels when titles contain rare terms (MLOps, PySpark, Kubernetes).
- * Corpus of 8,280 documents; top-500 candidates returned per query.

Technique 2 -- Dense Embeddings + FAISS ANN [NLP / Embeddings lectures]

- * BAAI/bge-small-en-v1.5 (ONNX via fastembed) encodes jobs at index time to 384-dim vectors.
- * FAISS IndexFlatIP retrieves top-500 semantically similar jobs at query time.
- * Captures cross-vocabulary equivalence (e.g. 'machine learning' ~ 'AI engineer').
- * Query vector updated in-place by the Rocchio algorithm after each feedback round.

Benchmark (Recall@10 and NDCG@10 -- 3 test profiles, 8,280 real Kaggle jobs)

Ground truth: jobs matching user skills OR role keywords in title/description.

Method	ML	ML	Analyst	Analyst	MLOps	MLOps
	R@10	N@10	R@10	N@10	R@10	N@10
BM25 (sparse)	0.013	0.854	0.001	1.000	0.008	1.000
SBERT+FAISS (dense)	0.006	1.000	0.001	1.000	0.003	1.000
Hybrid (production)	0.006	0.605	0.001	0.943	0.004	0.784

Dense retrieval achieves NDCG@10 = 1.000 -- relevant jobs ranked at position 1. BM25 provides broader recall. Hybrid (0.7 x dense + 0.3 x BM25) used in production.

3. Adaptive Learning -- Rocchio Algorithm

After each accept/reject/skip round, the Rocchio algorithm updates the user's query vector in 384-dimensional embedding space:

$$\mathbf{q_new} = \mathbf{a} * \mathbf{q_orig} + \mathbf{b} * \text{mean}(\text{accepted}) - \mathbf{g} * \text{mean}(\text{rejected}) \quad [\mathbf{a}=1.0, \mathbf{b}=0.8, \mathbf{g}=0.1]$$

- * Each accepted/rejected job encodes to 384-dim and shifts the query vector.
- * Updated vector persisted to SQLite (JSON); used for all subsequent FAISS queries.
- * Measured: NDCG@10 improved from 0.500 to 1.000 after a single feedback round.
- * Results page shows round number, NDCG before/after, and improvement delta.

4. Data Pipeline

- * Source: Techmap 'international-job-postings-september-2021' (Kaggle). 10,000 records sampled; 8,280 unique after SHA-256 deduplication (17.1% dedup rate).
- * Schema: nested JSON fields parsed -- position.name -> title, orgCompany.nameOrg -> company, orgAddress.addressLine -> location, orgTags.CATEGORIES -> skills.
- * Streaming: APScheduler polls Adzuna API every 5 min. New postings deduplicated and appended to the live FAISS index without full rebuild.
- * Deduplication: 1,711 duplicates removed from 10,000-row sample (17.1%).

5. Test Persona Results

Persona	Key Constraints	Str.	Par.	Irr.	Top-10 Rel.	Explain (3 jobs)	Resume (2 jobs)	PASS?
Aisha (ML Pivoter)	No senior/staff titles	2	8	0	10/10	3/3	2/2	PASS
	No defense/military				PASS	PASS	PASS	
	Remote/Bay Area, \$140k+							
Marcus (New Grad)	No 3+ yr exp required	3	7	0	10/10	3/3	2/2	PASS
	No contract/temp				PASS	PASS	PASS	
	Any US location, \$80k+							
Priya (Sr. MLOps)	No junior titles	1	8	1	9/10	3/3	2/2	PASS
	NYC/Remote, \$200k+				PASS	PASS	PASS	
	Senior roles only							
Kenji (Res./Visa)	No contract/temp (visa)	3	2	5	5/10	3/3	2/2	PASS
	US only, \$120k+				NOTE*	PASS	PASS	
	Research-focused							

* Kenji ranks 5-10 show noise from the 2021 Kaggle dataset's limited research/AI postings. Top-3 include Sr. ML Engineer (100% semantic match) and AI Engineer -- both strong matches. No contract roles confirmed. US-only filter verified active.

Capability Coverage -- All 6 Required Capabilities

#	Capability	Verification
1	Job Data Ingestion & Streaming	8,280 real Kaggle jobs; 1,711 dupes removed; Adzuna streaming configured
2	Profile Intake & Skill Matching	PDF + form upload; skills extracted from description text; filters active
3	Embedding-Based Retrieval	BAAI/bge-small-en -> FAISS; 93-100% semantic scores observed in QA testing
4	Multi-Stage Ranking + Metric	Hard filters -> BM25+FAISS -> hybrid re-rank; NDCG@10 benchmarked in UI
5	Adaptive Learning	Rocchio; NDCG 0.500->1.000 after 1 round; metrics shown in results page
6	Download Top Jobs	CSV: 51 rows with title/company/location/salary/URL -- verified all 4 personas

6. Limitations

- * Salary data: Kaggle dataset has no structured salary fields. Adzuna API provides structured salary for live postings.
- * Dataset scope: 2021 general jobs archive. Niche roles show lower relevance in long tail. Tech-filtered dataset would improve ranking quality.
- * External URLs: All URLs reflect September 2021 and are no longer active. Application stores and exports them correctly.
- * Embedding throughput: ~80ms/text on CPU; 50k-job index rebuild ~60 min. GPU inference would reduce to under 5 minutes.
- * Session persistence: SQLite resets on Render redeploy. Production deployment would use PostgreSQL with a persistent volume.

7. AI Tools

Developed using Claude Code (claude-sonnet-4-6) as the primary development assistant. Full prompt log in prompts.md. AI-generated code reviewed, tested, and modified throughout. Key prompts: system architecture, ranking pipeline, Rocchio learning, Techmap schema parser, Flask session architecture, and explainability panel. All benchmark results, persona validation, and architectural decisions reflect original engineering judgment.